KSP IntelHub

Karnataka State Police SCRB

AI Intelligence Platform

Complete Project Report

11 ML Models | 26 Database Tables | Full Architecture

Prepared for: HOD Presentation

Date: August 4, 2026

Competition: KSP Datathon 2026

GitHub: github.com/kartikbhartiya/Datathon

. Table of Contents

1. Project Overview & Problem Statement
2. System Architecture
3. Database Design - 26 Relational Tables
4. ML Model 1: Crime Volume Forecasting (Ridge + Fourier)
5. ML Model 2: Geospatial Crime Clustering (DBSCAN + Haversine)
6. ML Model 3: Co-Offender Link Prediction (Adamic-Adar)
7. ML Model 4: FIR Narrative Classifier (TF-IDF + Logistic Regression)
8. ML Model 5: Recidivism Risk Index (Multi-Factor Scoring)
9. ML Model 6: Case Allocation Predictor (Random Forest)
10. ML Model 7: Suspect Conviction Predictor (Random Forest)
11. ML Model 8: Crime Community Detection (Louvain Algorithm)
12. ML Model 9: Suspect Recommendation Engine
13. ML Model 10: Socio-Economic Crime Correlation (Pearson)
14. ML Model 11: Suspect Crime Correlation Analysis
15. AI Copilot - LLM Investigation Assistant
16. Technology Stack Summary
17. API Endpoints Reference
18. Key Talking Points for the Presentation

1. Project Overview & Problem Statement

What is KSP IntelHub?

KSP IntelHub is an AI-driven crime intelligence and analytical platform built for the State Crime Records Bureau (SCRB) of the Karnataka State Police (KSP). It replaces manual, siloed Excel-based crime reporting with an automated, real-time intelligence dashboard powered by machine learning.

Built for the KSP Datathon 2026, a competition organized by Karnataka State Police. The platform is designed for SCRB officers and District SPs to gain real-time situational awareness.

The 4 Core Problems It Solves

Problem 1: Data Silos & Manual Reporting

Solution: Unified 26 relational database tables (FIRs, Accused, Courts, Vehicles, Cybercrime, Officers, etc.) into a single queryable intelligence core.

Problem 2: Lack of Advanced Analytics

Solution: Integrated 11 ML models - Ridge regression forecasting, DBSCAN clustering, Adamic-Adar link prediction, TF-IDF NLP classification, Random Forest predictors, Louvain community detection, and more.

Problem 3: Reactive vs. Proactive Policing

Solution: Automated patrol dispatch recommendations based on predicted weekly crime spikes and high-risk DBSCAN zones.

Problem 4: Opaque Criminal Networks

Solution: 3D force-directed network visualization with link prediction to uncover hidden co-offender alliances across Rowdy-Sheeters, cyber fraud rings, and stolen vehicle networks.

2. System Architecture

Architecture Overview

```
USER (Browser)
     |
     v
FRONTEND - Next.js 16 / React 19 (Port 3000)
  Pages: Overview, Cases, Map, Network, Analytics,
         Intel, Officers, Dispatch, Suspects, Courts,
         Cyber, Vehicles, Missing Persons
  Tech:  Tailwind CSS v4, Framer Motion, Recharts,
         React Force Graph, Google Maps JS API
     |
     v  (REST API calls)
BACKEND - Python Flask (Port 9000)
  main.py       -> Flask routes, NetworkX graph engine
  analytics.py  -> 11 ML models pipeline
  Data: 26 CSV files loaded into Pandas DataFrames
  Graph: NetworkX for relational network analysis
  LLM: Groq (LLaMA 3.3 70B) / NVIDIA (MiniMax M3)
     |
     v
DEPLOYMENT - Zoho Catalyst (AppSail Docker + Web Hosting)
```

How Data Flows

- Startup: Flask server loads all 26 CSV data files into Pandas DataFrames and builds a NetworkX graph connecting suspects, cases, officers, cyber IDs, and vehicles.
- ML Training: On boot, the server trains all ML models (classifiers, forecasters) using the loaded data. Models are cached in memory.
- API Request: Frontend makes REST calls -> Flask routes call analytics.py functions -> return JSON results.
- AI Copilot: User queries go to /api/chat -> system dynamically searches all 26 tables in memory -> constructs context prompt -> sends to Groq/NVIDIA LLM -> returns intelligent response.

3. Database Design - 26 Relational Tables

The platform integrates data from 26 tables/registries loaded from CSV files (simulating what would be a live SQL database in production):

Table Name	Purpose	Key Fields
CaseMaster	Core FIR/case records	CaseMasterID, CrimeNo, lat/lng, BriefFacts
Accused	Accused persons per ca	AccusedMasterID, CaseMasterID, PersonID
Victim	Victims per case	VictimMasterID, CaseMasterID, VictimName
CriminalsRegistry	Habitual offender DB	PersonID, gang, risk, rowdy_sheeter
suspects	Suspect profiles	suspect_id, name, gang, risk_level
case_links	Case-suspect relations	case_id, suspect_id, relationship
case_reports	FIR narrative reports	case_id, report_text, officer_id
CourtCasesRegistry	Court proceedings	CaseMasterID, conviction_status
ChargesheetDetails	Chargesheet filings	CSID, CaseMasterID, csdate
cybercrime	Digital fraud details	case_id, upi_ids, bank_accounts, fraud_amount
stolen_vehicles	Vehicle theft registry	vehicle_number, recovery_status
missing_persons	Missing persons	person_id, name, found_status
OfficersRegistry	Officer performance	EmployeeID, closure_rate
Employee	Officer details	EmployeeID, RankID, AppointmentDate
complaints	Dial-112 logs	complaint_id, emergency_type
cctv	CCTV feed metadata	camera_id, detected_faces/vehicle
socioeconomic	District socio-eco dat	district, unemployment, literacy
Act/Section	Legal references	ActCode, SectionCode, Description
ActSectionAssociation	Acts per case	CaseMasterID, ActID, SectionID
ArrestSurrender	Arrest records	ArrestSurrenderID, CaseMasterID
Court	Court master data	CourtID, CourtName, DistrictID
CrimeHead/SubHead	Crime classification	Crime groups and sub-categories
District/State	Geographic hierarchy	District and state master tables
Designation/Rank	Officer rank hierarchy	RankName, Hierarchy
Unit/UnitType	Police station org	Station types, hierarchy

Terminology: A "Rowdy Sheeter" is a legal classification under Sections 95 & 96 of the Karnataka Police Act for habitual offenders involved in violence, extortion, or gang activities. It is a boolean flag in the CriminalsRegistry.

4. ML Model 1: Crime Volume Forecasting

Algorithm: Ridge Regression + Fourier Seasonality Features

File: analytics.py -> _forecast_one() and build_forecast()

What It Does

Predicts weekly crime incident counts up to 8 weeks into the future at three levels:

- State-wide (all of Karnataka)
- Per-district (each of Karnataka's districts)
- Per crime type (Murder, Cyber Fraud, Vehicle Theft, etc.)

Step-by-Step Workflow

```
Raw FIR data (timestamps)
  -> Group by week (Pandas resample "W")
  -> Build feature matrix X:
       Linear trend: t/n (normalised time index)
       Fourier terms:
         sin(2*pi*t/52), cos(2*pi*t/52) - annual cycle
         sin(2*pi*t/26), cos(2*pi*t/26) - semi-annual
         sin(2*pi*t/13), cos(2*pi*t/13) - quarterly
  -> Fit RidgeCV (alphas: 0.1, 1.0, 5.0, 10.0, 50.0)
  -> Compute residuals -> sigma (std deviation)
  -> Predict next 8 weeks:
       Point forecast = model.predict(future features)
       Lower = forecast - 1.28*sigma (80% CI)
       Upper = forecast + 1.28*sigma (80% CI)
  -> Trend detection:
       >6% increase -> "rising"
       <-6% decrease -> "falling"
       Otherwise -> "stable"
```

Key Mathematical Concepts

Fourier Series:

Any periodic function can be decomposed into sine and cosine waves. By using sin(2*pi*t/P) and cos(2*pi*t/P) for different periods P, we capture seasonal patterns in crime data without needing a dedicated time-series library.

Ridge Regression:

Linear regression with L2 penalty (lambda * ||beta||^2) that shrinks coefficients to prevent overfitting. The cross-validated alpha selection (RidgeCV) automatically finds the best regularization strength from [0.1, 1.0, 5.0, 10.0, 50.0].

Prediction Interval:

1.28*sigma corresponds to the 80th percentile of a normal distribution. +/-1.28 standard deviations covers approximately 80% of expected outcomes.

Why Ridge + Fourier (Not ARIMA or Prophet)?

- ARIMA requires stationarity testing and parameter tuning (p,d,q) which is complex for 30+ district-level series.
- Prophet is heavy and slow for batch forecasting across many series.
- Ridge + Fourier is lightweight, fast, and gives comparable accuracy for weekly aggregates.
- Regularization prevents overfitting on small district-level datasets.

5. ML Model 2: Geospatial Crime Clustering

Algorithm: DBSCAN with Haversine Distance Metric

File: analytics.py -> build_geo_clusters()

What It Does

Groups geographically dense crime incidents into bounded risk zones (clusters) on the map, identifying crime hotspots with center coordinates, radius in km, dominant crime type, average severity, and open case count.

Step-by-Step Workflow

```
Filter FIR data: severity_score >= 40 (high-severity only)
  -> Convert lat/lng to radians
  -> Run DBSCAN:
       eps = 45km / 6371km (Earth radius) = 0.00707 radians
       min_samples = 3
       metric = "haversine"
  -> For each cluster (label != -1 noise):
       Centroid = mean(lat, lng)
       Radius = 90th-percentile haversine distance
       Dominant crime = mode(crime_type)
       Avg severity = mean(severity_score)
  -> Sort by case count, return top 20
```

The Haversine Formula

```
a = sin^2(delta_lat/2) + cos(lat1) * cos(lat2) * sin^2(delta_lng/2)
distance = 2R * arcsin(sqrt(a))
where R = 6371.0088 km (Earth's mean radius)
```

This is critical because Euclidean distance does not work for geographic coordinates. 1 degree of longitude varies in km depending on latitude.

Why DBSCAN (Not K-Means)?

- Does NOT require pre-specifying k (number of clusters) unlike K-Means.
- Handles noise - isolated incidents are classified as noise (label = -1) and excluded.
- Finds arbitrary-shaped clusters - crime zones are not always circular.
- Works natively with haversine metric - essential for geographic data.
- eps = 45 km chosen based on Karnataka district sizes for meaningful clusters.

6. ML Model 3: Co-Offender Link Prediction

Algorithm: Adamic-Adar Index on Co-Offender Network Graph

File: analytics.py -> build_link_prediction()

What It Does

Predicts hidden criminal alliances - identifies pairs of suspects who don't yet share a case but are likely connected based on shared network neighbors. "Who is likely to team up next for a crime?"

Step-by-Step Workflow

```
Step 1: Build undirected co-offender graph G
  Nodes = all suspects
  Edges: suspect A & B in same Case -> edge(A, B)
  Edges: relationship = "Known Associate" -> edge(A, B)

Step 2: Find candidate non-edges (friend-of-friend pairs)
  For each node u -> neighbors of neighbors
  If u and x NOT directly connected but share a neighbor
  -> candidate pair

Step 3: Score with Adamic-Adar Index
  AA(u,v) = SUM over w in common_neighbors(u,v):
              1 / log(degree(w))
  Intuition: a shared associate who knows few people
  is MORE significant than one who knows everyone

Step 4: Rank by score, return top 12 predictions
```

Why Adamic-Adar (Not Just Common Neighbors)?

Common Neighbors just counts shared connections. Adamic-Adar weights by rarity: a shared associate who only knows 2 people is far more indicative of a real connection than one who knows 200 people. Named after Lada Adamic and Eytan Adar (2003) for social network prediction.

Real-World Use Case

Suspect A is part of a car theft ring. Suspect B is part of a different theft ring. They don't share any FIR cases. But both know Suspect C (a fence who sells stolen goods). Adamic-Adar scores this pair highly, alerting police to a potential emerging alliance.

7. ML Model 4: FIR Narrative Crime Classifier

Algorithm: TF-IDF Vectorizer + Logistic Regression

File: analytics.py -> train_crime_classifier()

What It Does

Takes the raw text of an FIR narrative (the "Brief Facts" written by police) and automatically classifies it into a crime category (Murder, Cyber Fraud, Vehicle Theft, etc.) with >90% validation accuracy.

Step-by-Step Workflow

```
Step 1: Merge case_reports with FIRs on case_id
Step 2: TF-IDF Vectorization
  stop_words = "english" (remove the, is, and, etc.)
  ngram_range = (1,2) (single words AND 2-word phrases)
  min_df = 3 (ignore words in < 3 documents)
  max_features = 8000
Step 3: 80/20 stratified train-test split
Step 4: Logistic Regression (C=10.0, max_iter=1000)
Step 5: Evaluate accuracy + macro F1-score
Step 6: Refit on ALL data for production predictor
Step 7: Extract top 8 keywords per class (Explainable AI)
```

What is TF-IDF?

TF-IDF = Term Frequency x Inverse Document Frequency

```
TF(t,d) = count(term t in doc d) / total_terms(d)
IDF(t)  = log(total_docs / docs_containing_t)
TF-IDF  = TF * IDF

TF: How important is this word in THIS document?
IDF: How rare is this word across ALL documents?
"accused" appears in every FIR -> low IDF -> low score
"snatched chain" appears only in snatching FIRs -> high score
```

Why Logistic Regression (Not Deep Learning)?

- Works extremely well with TF-IDF features
- Interpretable: Can extract which words drive each prediction (Explainable AI)
- Fast to train and predict (no GPU needed)
- With C=10.0 and 8000 features, achieves >90% accuracy

Explainability: Class Keywords Example

```
Cyber Fraud: ["phishing","bank account","upi","fraud","otp"]
Murder: ["stabbed","deceased","knife","blood","killed"]
Vehicle Theft: ["vehicle","stolen","parked","registration"]
```

8. ML Model 5: Recidivism Risk Index

Algorithm: Transparent Multi-Factor Weighted Scoring (0-100)

File: analytics.py -> build_recidivism()

What It Does

Calculates a recidivism risk score (0-100) for every offender, predicting their likelihood of re-offending. Classified into bands: Extreme (>=75), High (>=55), Elevated (>=35), Moderate (<35).

The Scoring Formula

```
Risk Score = (prior_offences * 30) + (prior_arrests * 20)
           + (risk_level * 20) + (network_reach * 12)
           + (active_warrant * 10) + (gang_ties * 8)
                                        Total = 100
```

Factor	Weight	Calculation
Prior Offences	30	past_fir_count / 10 (capped at 10)
Prior Arrests	20	past_arrests / 8 (capped at 8)
Risk Level	20	Critical=1.0, High=0.75, Medium=0.45, Low=0.2
Network Reach	12	known_associates count / 6 (capped at 6)
Active Warrant	10	1.0 if active warrant exists, else 0.0
Gang Ties	8	1.0 if affiliated to any gang, else 0.0

Why Transparent Scoring (Not Black-Box ML)?

- Legal defensibility: Police decisions must be explainable in court.
- Auditability: Every officer can trace why a suspect scored 82/100.
- No training data bias: Does not learn discriminatory patterns from historical data.
- Domain expert alignment: Weights based on criminological research.

9. ML Model 6: Case Allocation Predictor

Algorithm: Random Forest Classifier (100 trees, max_depth=8)

File: analytics.py -> train_allocation_predictor()

What It Does

Predicts the probability of successful case closure if a specific officer is assigned to a specific case. Optimizes officer-case matching for maximum solve rates.

Features Used (11 features)

Feature	Source	Description
severity_score	Case	How severe the crime is (0-100)
fatalities	Case	Number of deaths
injured	Case	Number of injuries
witnesses	Case	Number of witnesses
closure_rate	Officer	Historical case closure rate (0-1)
cases_assigned	Officer	Number of active cases
commendations	Officer	Departmental commendations
experience_years	Derived	Years since appointment date
spec_match	Derived	1.0 if specialization matches crime
crime_encoded	Case	Label-encoded crime type
rank_encoded	Officer	Label-encoded officer rank

Why Random Forest?

- Handles mixed feature types (numerical + categorical).
- Built-in feature importance ranking.
- Robust to outliers; no feature scaling needed.
- Ensemble of 100 trees gives stable predictions.

10. ML Model 7: Suspect Conviction Predictor

Algorithm: Random Forest Classifier (50 trees, max_depth=5)

File: analytics.py -> train_suspect_conviction_model()

What It Does

Predicts the likelihood of conviction for an accused person based on their profile and criminal history. Helps prosecutors prioritize cases with strong evidence.

Features Used (7 features)

- age - Age of the accused
- gender - Gender ID
- past_firs - Number of prior FIR records
- past_arrests - Number of prior arrests
- rowdy_sheeter - Is this person a registered Rowdy Sheeter?
- history_sheet - Does this person have an opened history sheet?
- active_warrant - Is there an outstanding warrant?

Training Process

1. Merge Accused table -> CriminalsRegistry -> CourtCasesRegistry
2. Filter to cases with known verdict: Convicted or Acquitted
3. Target variable: conviction_status == "Convicted" (binary 0/1)
4. Train Random Forest (50 trees, max_depth=5)
5. Output: Feature importances showing which factors predict conviction

11. ML Model 8: Crime Community Detection

Algorithm: Louvain Algorithm (Community Detection)

File: analytics.py -> detect_crime_communities()

What It Does

Automatically detects organized crime cells/syndicates by finding densely connected communities in the co-offender network graph. Reveals hidden gang structures not explicitly recorded.

How It Works

```
Step 1: Build undirected graph from edges where
        relationship in {Co-Offender, Charged, Known Associate}
Step 2: Apply Louvain community detection
        Maximizes modularity Q (density within vs between)
        Fallback: Label Propagation if Louvain fails
Step 3: Filter communities >= 3 members
Step 4: Score community risk:
        Low=1, Medium=2, High=3, Critical=4
        >=3.2 = Critical, >=2.5 = High, >=1.5 = Medium
Step 5: Return syndicates with IDs, size, members, gangs
```

The Louvain algorithm is a greedy optimization algorithm running in O(n log n) time. It maximizes modularity Q - a measure of how much the network structure deviates from a random network. Proposed by Blondel et al. (2008).

12. ML Model 9: Suspect Recommendation Engine

Algorithm: Multi-Factor Scoring (MO + Spatial + Risk)

File: analytics.py -> recommend_suspects_for_case()

What It Does

Given an unsolved case, recommends the top 5 most likely suspects based on three scoring dimensions:

1. Modus Operandi (MO) Similarity - Max 40 points

- Matching crime type history: +20 points
- Matching weapon used: +10 points
- Matching vehicle used: +5 points
- Matching target type: +5 points

2. Spatiotemporal Proximity - Max 35 points

- Operated in same district: +15 points
- Crime scene < 5 km from past activity: +5 to +20 (inverse distance)
- Crime scene < 15 km from past activity: +10 points

3. Offender Risk Profile - Max 25 points

- Flagged as Rowdy Sheeter: +8 points
- Active arrest warrant: +8 points
- Opened history sheet: +4 points
- Past arrests (x1.5 per arrest, max 5): up to +5 points

Spatial proximity uses the Haversine formula to calculate minimum distance between the crime scene and all of the suspect's past locations.

13. ML Model 10: Socio-Economic Crime Correlation

Algorithm: Pearson Correlation Analysis

File: analytics.py -> build_socioeconomic_correlation()

What It Does

Correlates district-level crime patterns with socio-economic indicators. Answers the "why behind the where" - why certain districts have more crime.

Correlation Pairs Analyzed

Socio-Economic Factor	Crime Metric	Question
Unemployment	Property-crime %	Does unemployment drive theft?
Unemployment	Crime rate/100k	Does unemployment drive crime?
Urbanisation	Cyber-fraud %	Are urban areas more cyber-prone?
Literacy	Cyber-fraud %	Do literate areas see more fraud?
Per-capita income	Crime rate/100k	Does income reduce crime?
Police/100k	Crime rate/100k	Does more policing reduce crime?
Slum population	Property-crime %	Do slums drive property crime?
Youth share	Crime rate/100k	Does younger population = more?

Pearson Correlation Coefficient

```
r = SUM((xi - x_mean)(yi - y_mean)) /
    (sqrt(SUM(xi - x_mean)^2) * sqrt(SUM(yi - y_mean)^2))

|r| >= 0.6  -> "strong"
|r| >= 0.35 -> "moderate"
|r| <  0.35 -> "weak"
r > 0  -> positive (both increase together)
r < 0  -> negative (one up, other down)
```

14. ML Model 11: Suspect Crime Correlation

Algorithm: Cosine Similarity + Z-Score Deviance

File: analytics.py -> get_suspect_crime_correlation()

What It Does

Generates a detailed criminological profile for a single suspect:

- Temporal Correlation: Cosine similarity between suspect's time-of-day distribution and system-wide distribution.
- Socio-Economic Profile: Weighted average of indicators across suspect's active districts.
- Severity Deviance: Z-score = (suspect_avg - system_avg) / system_std.
- Crime Specialization: >70% in one type = "High Specialist", <40% = "Generalist".
- Co-Offender Network: Who does this suspect work with?

15. AI Copilot - LLM Investigation Assistant

Model: Groq LLaMA 3.3 70B / NVIDIA MiniMax M3

Endpoint: /serverless/api/chat (POST)

What It Does

A natural language investigation assistant allowing officers to ask complex questions in English or Kannada:

- "Who are the top Rowdy Sheeters in Bengaluru with active warrants?"
- "Show me all cyber fraud cases with fraud amount > 50 lakhs"
- "Which districts saw a spike in vehicle thefts this quarter?"

Architecture (RAG-like)

```
User Query (text or voice)
  -> Dynamic Memory Search:
       Search all 26 in-memory DataFrames
       Filter relevant rows by query keywords
  -> Construct System Prompt:
       Database schema + matched records + domain terms
  -> Send to LLM (Groq / NVIDIA API)
       Model: llama-3.3-70b-versatile
  -> Return natural language response
  -> Optional: Smart routing (e.g., /network?q=name)
```

Why RAG-like (Not Fine-Tuned)?

- No training data needed - works with any new dataset immediately.
- Always up-to-date - queries live data, not a static model.
- Flexible - handles questions never explicitly trained on.
- Cost-effective - uses API calls, no GPU infrastructure.

16. Technology Stack Summary

Backend (Python Flask API)

Library	Version	Purpose
Flask	3.0.3	REST API framework
Pandas	2.2.2	Data loading, manipulation, aggregation
NumPy	1.26.4	Numerical computations
Scikit-learn	1.4.2	ML: Ridge, DBSCAN, LogReg, RF, TF-IDF
NetworkX	3.3	Graph: Adamic-Adar, Louvain community
OpenAI	1.40.0	LLM API client (Groq/NVIDIA endpoints)
Gunicorn	22.0.0	Production WSGI server

Frontend (Next.js Dashboard)

Library	Version	Purpose
Next.js	16.2.9	React framework with App Router
React	19.2.4	UI component library
Tailwind CSS	4.x	Utility-first CSS framework
Framer Motion	12.40.0	Animations and transitions
Recharts	3.8.1	Charts and data visualization
Force Graph 2D/3D	1.29.1	Network graph visualization
Three.js	0.184.0	3D rendering for network graphs
Google Maps API	-	Geospatial heatmaps and markers

17. API Endpoints Reference

Method	Endpoint	Description
GET	/serverless/api/health	System health, record counts, graph stats
GET	/serverless/api/stats	KPI aggregations & crime trends
GET	/serverless/api/cases	Paginated FIR list with search/filters
GET	/serverless/api/network	Network graph nodes + edges
GET	/serverless/api/heatmap	Spatial coords, KDE, DBSCAN zones
GET	/serverless/api/analytics	Full ML predictions (all models)
POST	/serverless/api/chat	AI Copilot NL query engine

POST	/serverless/api/classify	FIR narrative crime classifier

18. Key Talking Points for the Presentation

Opening (2 minutes)

"KSP IntelHub is an AI-powered crime intelligence platform for Karnataka State Police that replaces manual Excel-based reporting with real-time ML-driven analytics across 26 integrated data registries."

The ML Story (5-7 minutes)

Walk through these 8 key points:

- "We forecast crime" - Ridge + Fourier gives 8-week predictions with 80% confidence intervals
- "We find crime zones" - DBSCAN clusters geo-located incidents using Earth's curvature (Haversine)
- "We predict hidden alliances" - Adamic-Adar on the co-offender NetworkX graph
- "We auto-classify FIRs" - TF-IDF + Logistic Regression at >90% accuracy with explainable keywords
- "We score recidivism risk" - Transparent 100-point weighted formula (legally defensible)
- "We detect crime syndicates" - Louvain community detection finds hidden gang structures
- "We recommend suspects" - Multi-factor scoring with MO matching and spatial proximity
- "We correlate crime with socio-economics" - Pearson analysis reveals why certain districts are hotspots

If HOD Asks Technical Questions

Why Ridge over ARIMA?

Fourier features capture seasonality without stationarity requirements; Ridge's L2 regularization handles small district-level datasets.

Why DBSCAN over K-Means?

No need to specify k; handles noise; works with haversine metric.

Why Logistic Regression over Neural Networks?

Interpretable (extractable keywords); fast; >90% accuracy is sufficient for this use case.

Why scoring formula for recidivism?

Legal defensibility and auditability; no black-box bias risks.

Graph Analytics:

NetworkX provides O(n log n) Louvain community detection and efficient Adamic-Adar computation.

Closing (1 minute)

"The platform is deployed on Zoho Catalyst, integrates 11 ML models, processes 26 data registries, and gives Karnataka Police officers proactive intelligence instead of reactive reports."